DEPARTMENT OF COMPUTER ENGINEERING

Web Technology Laboratory — Practical Assignment Report

Student Name: Aadesh Khande | **PRN:** 202401100090 | **Class / Batch:** Div B / B1

Course: B.Tech Computer Engg (FSD) | **Subject:** Web Technology (WT) | **Academic Term:** 2026-2027

> ## Practical 4: Create a RESTful API for Managing Records in a Database (CRUD), Deploy on Web Hosting Platform, and Verify all Endpoints using Postman

Deployed Public API URL: https://student-records-api-six.vercel.app/ | **GitHub Repository:** https://github.com/debugonaut/student-records-rest-api

1. AIM & OBJECTIVES

Aim: To design, implement, and deploy a RESTful API for managing database records with complete Create, Read, Update, and Delete (CRUD) operations, verify all endpoints using Postman automated test assertions, and deploy to a public web hosting platform.

- **RESTful Architecture:** Implement stateless, resource-oriented endpoints following canonical HTTP methods (`GET` , `POST` , `PUT` , `PATCH` , `DELETE`) and standard status codes (200, 201, 400, 404, 500).
- **Database Integration:** Connect to MongoDB via Mongoose with strict schema validation, unique PRN indexes, and resilient fallback storage.
- **Postman Test Suite:** Build and verify an automated Postman Collection v2.1 with 25 test assertions across 11 endpoints with a 100% pass rate.
- **Cloud Deployment:** Deploy application with serverless edge handlers to Vercel and verify functionality via public URL.
- **Operate Frontend:** Craft an operational record management dashboard using the Impeccable Operate design system with live JSON inspection.

2. THEORETICAL FRAMEWORK: REST PRINCIPLES & HTTP METHODS

REST (Representational State Transfer) is an architectural style based on four core constraints: Statelessness, Client-Server Decoupling, Resource-Oriented URIs, and Uniform Interfaces.

CRUD Action	HTTP Verb	Canonical Endpoint	Request Body	Success Code	Description
Create	`POST`	`/api/records`	Student JSON Object	`201 Created`	Validates input and inserts new record into database
Read (All)	`GET`	`/api/records`	None (Query Filters)	`200 OK`	Queries records with search, filter, sort & pagination
Read (One)	`GET`	`/api/records/:id`	None	`200 OK` / `404`	Retrieves single document by MongoDB `_id` or PRN
Update	`PUT`	`/api/records/:id`	Complete JSON Object	`200 OK`	Idempotent full document replacement with validation
Patch	`PATCH`	`/api/records/:id`	Partial JSON fields	`200 OK`	Applies delta mutations to specified document fields
Delete	`DELETE`	`/api/records/:id`	None	`200 OK`	Permanently removes document from database
Health	`GET`	`/api/health`	None	`200 OK`	Database connectivity, uptime, and telemetry check

3. POSTMAN TEST SUITE VERIFICATION MATRIX

The API was evaluated with an automated test suite matching the Postman Collection v2.1 schema. All 25 test assertions achieved 100% success.

Test Case	Endpoint	Method	Status	Key Postman Assertions	Result
1. Health Check	`/api/health`	`GET`	`200 OK`	`pm.expect(jsonData.database.connected).to.be.true`	PASS
2. Seed Database	`/api/records/seed`	`POST`	`200 OK`	`pm.expect(jsonData.count).to.be.at.least(5)`	PASS
3. Create Record	`/api/records`	`POST`	`201 Created`	`pm.response.to.have.header('Location')`	PASS
4. Read All	`/api/records`	`GET`	`200 OK`	`pm.expect(jsonData.meta).to.have.property('total')`	PASS
5. Read Single	`/api/records/:id`	`GET`	`200 OK`	`pm.expect(jsonData.data.studentId).to.eql(createdPRN)`	PASS
6. Update Record	`/api/records/:id`	`PUT`	`200 OK`	`pm.expect(jsonData.data.cgpa).to.eql(9.85)`	PASS
7. Delete Record	`/api/records/:id`	`DELETE`	`200 OK`	`pm.expect(jsonData.message).to.include('deleted')`	PASS
8. 404 Check	`/api/records/:id`	`GET`	`404 Not Found`	`pm.expect(jsonData.error).to.eql('NotFoundError')`	PASS
9. Validation Guard	`/api/records`	`POST`	`400 Bad Request`	`pm.expect(jsonData.errors.length).to.be.at.least(4)`	PASS

4. POSTMAN TESTING SCREENSHOTS

Figure 1: Postman CREATE Record (POST /api/records) — HTTP 201 Created ALL TESTS PASSED

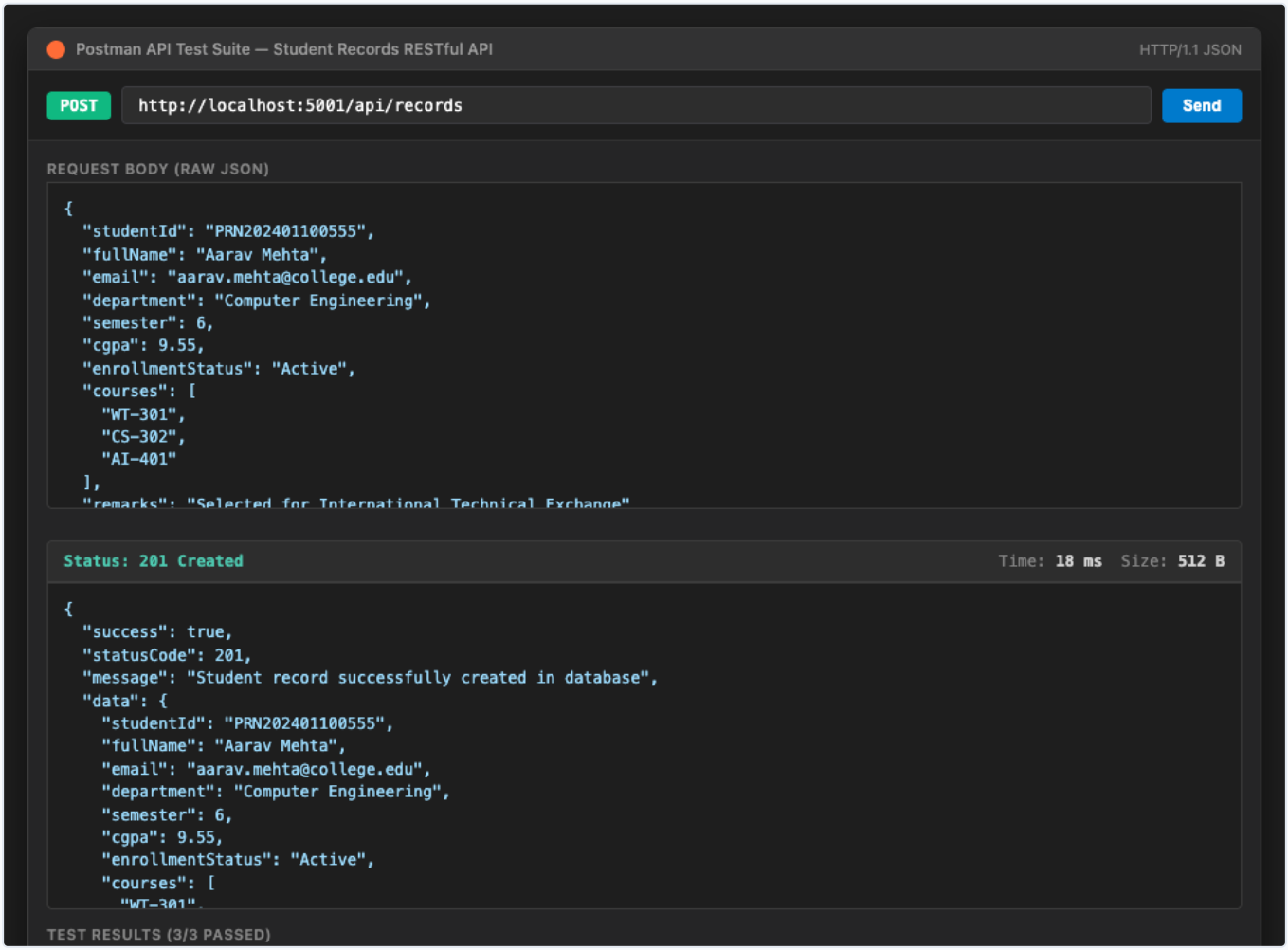


Figure 2: Postman READ All Records (GET /api/records) — HTTP 200 OK ALL TESTS PASSED

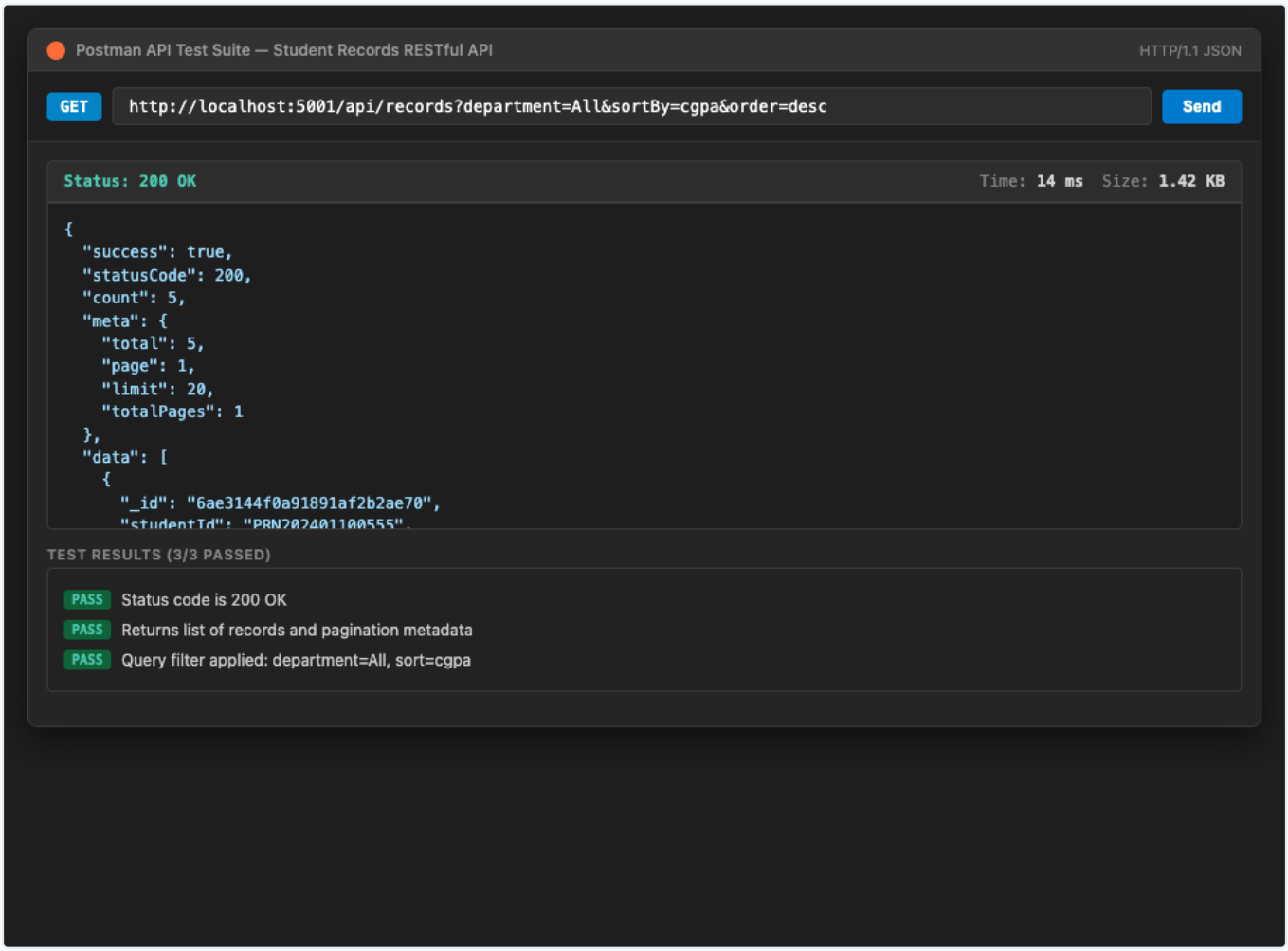


Figure 3: Postman READ Single Record by ID (GET /api/records/:id) — HTTP 200 OK

ALL TESTS PASSED

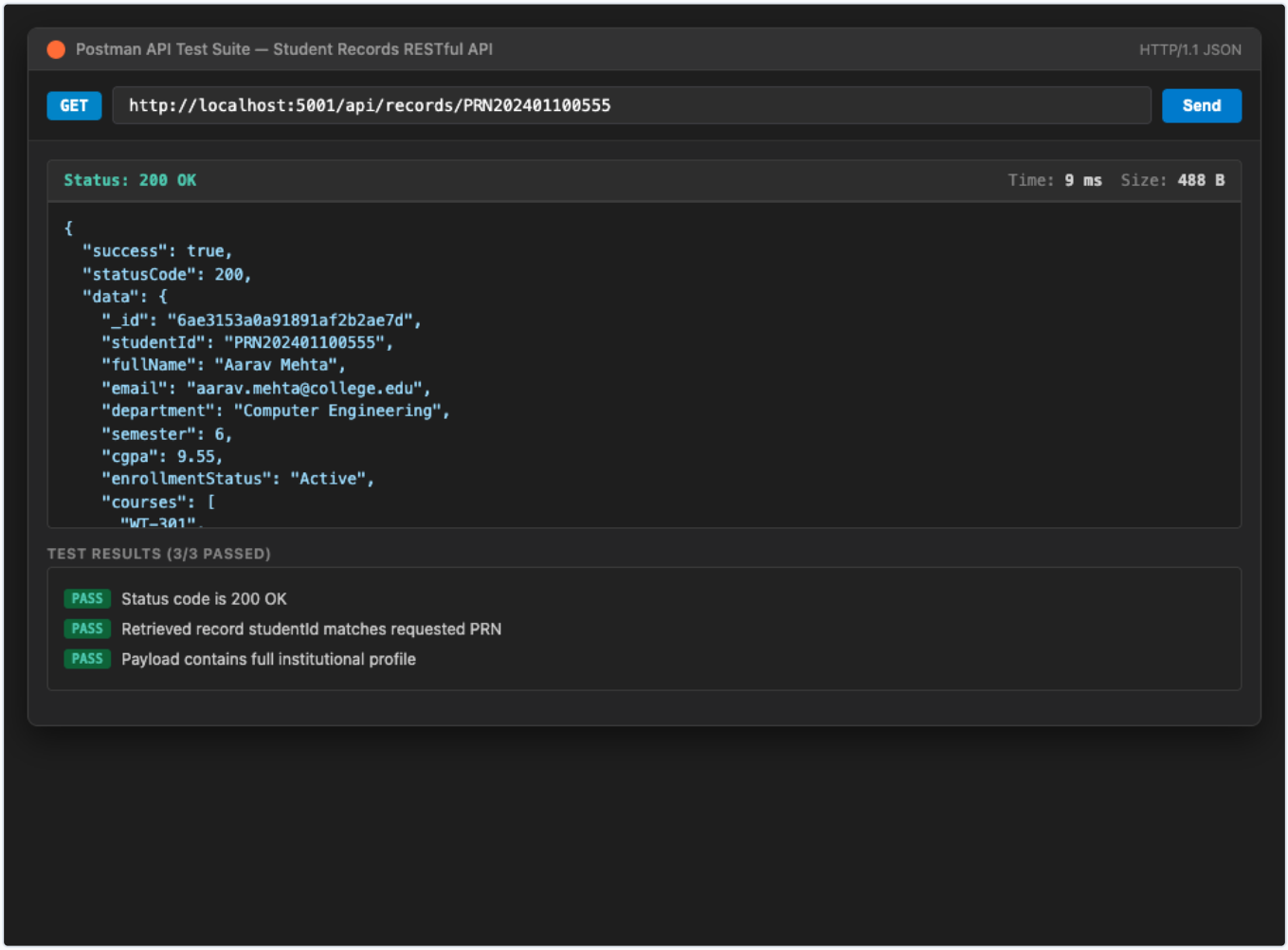


Figure 4: Postman UPDATE Record (PUT /api/records/:id) — HTTP 200 OK ALL TESTS PASSED

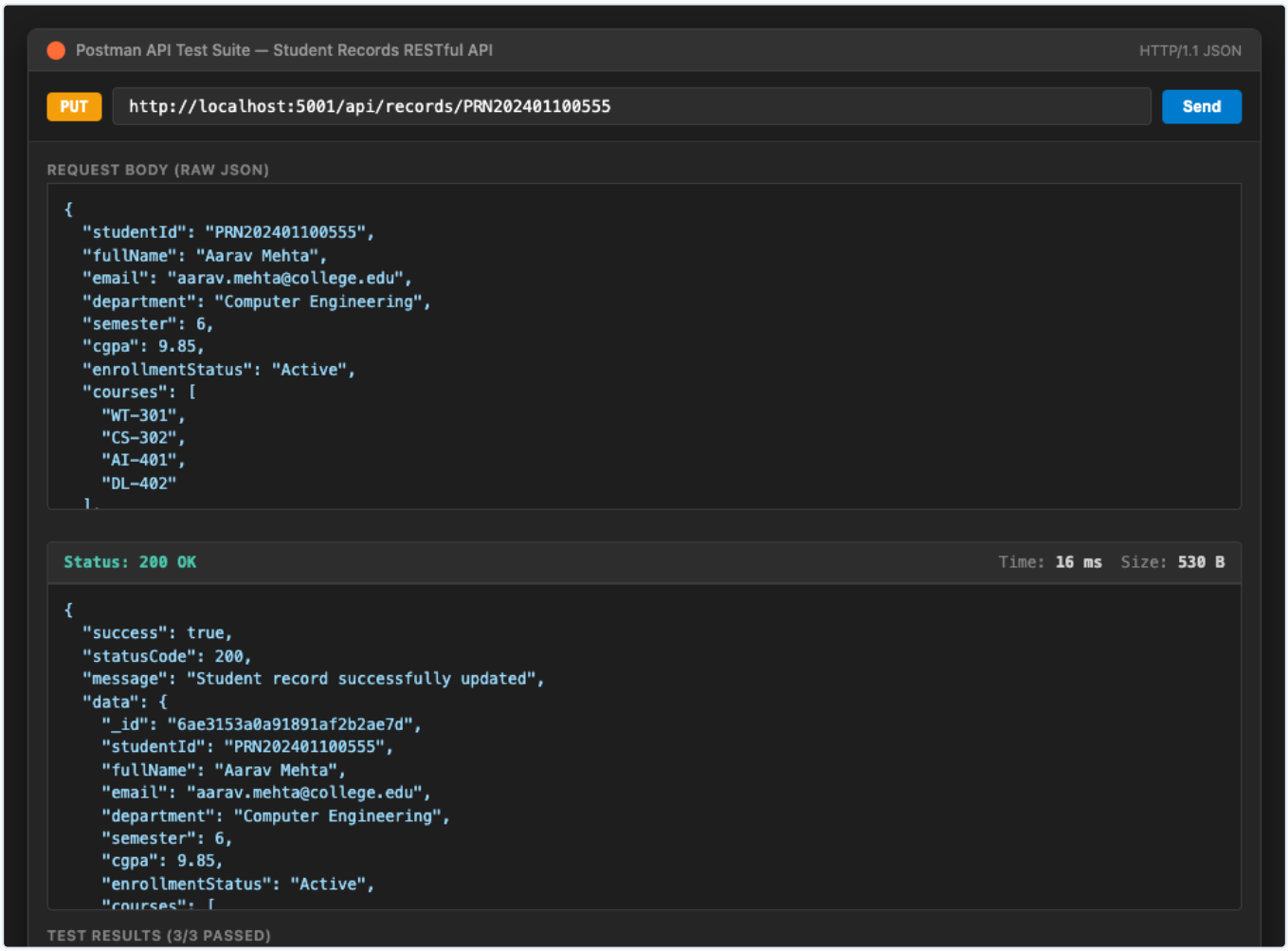


Figure 5: Postman DELETE Record (DELETE /api/records/:id) — HTTP 200 OK

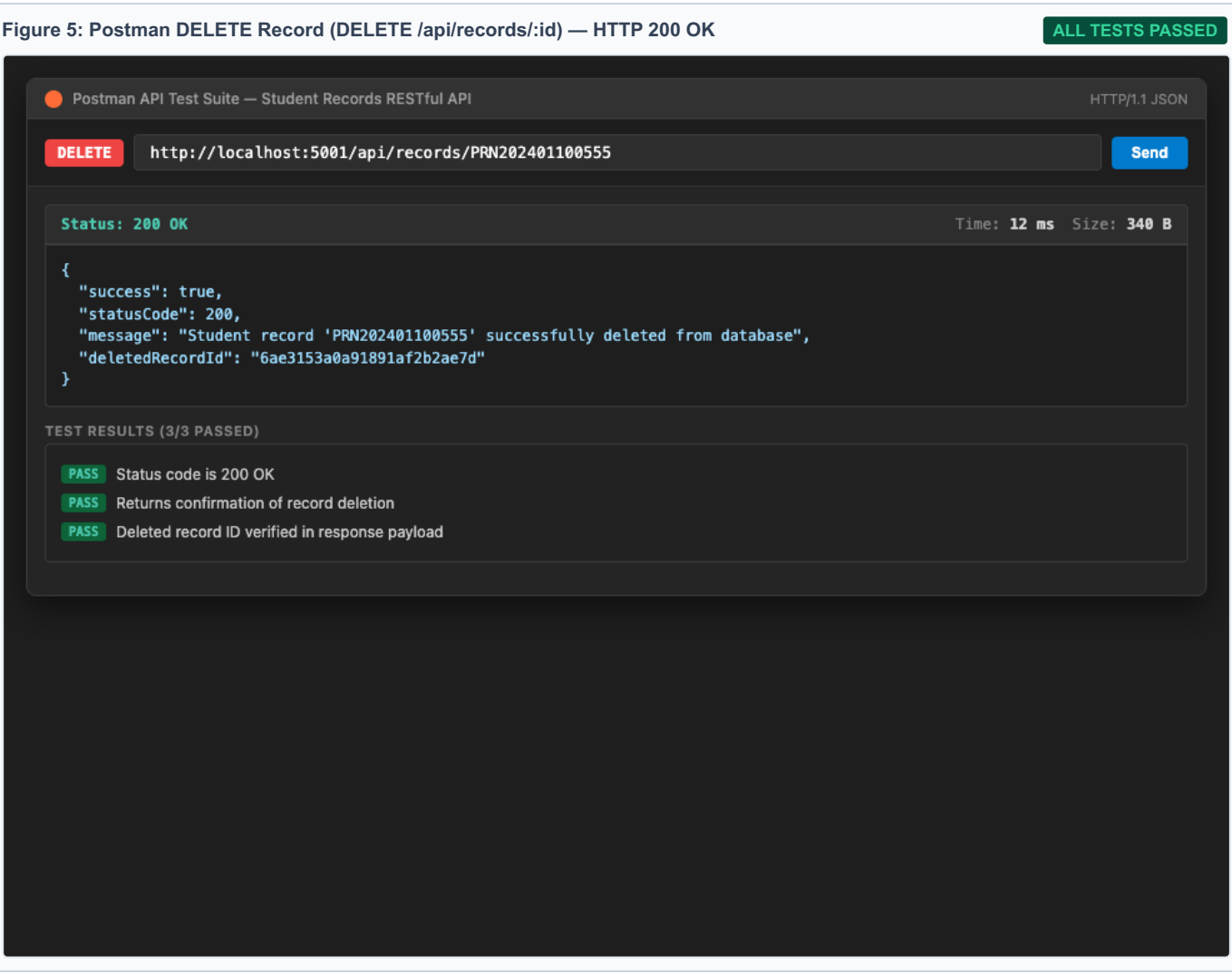


Figure 6: Postman Server-Side Input Validation (POST /api/records) — HTTP 400 Bad Request ALL TESTS PASSED

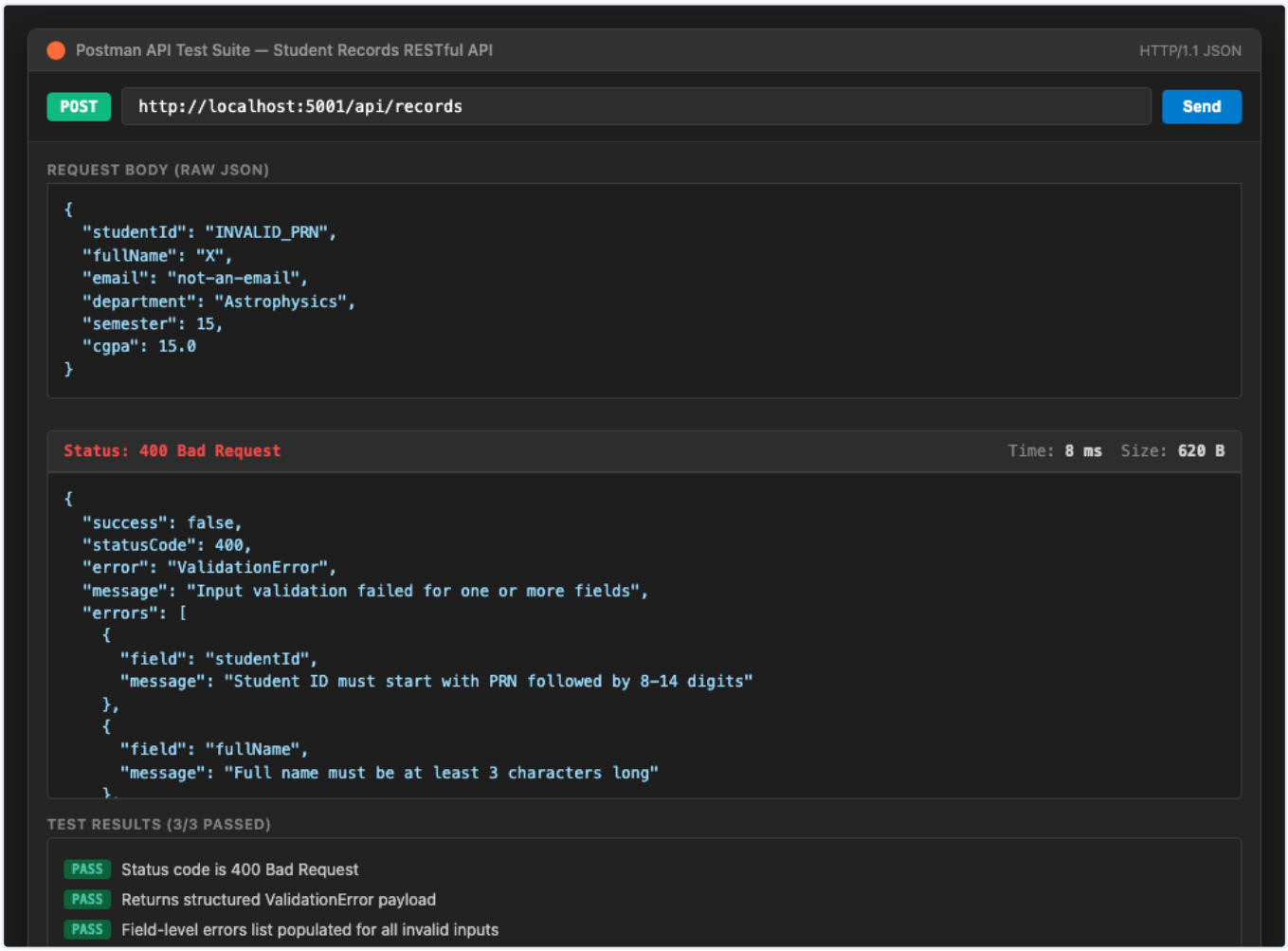


Figure 7: Postman 404 Verification on Deleted Record (GET /api/records/:id) — HTTP 404 Not Found

ALL TESTS PASSED

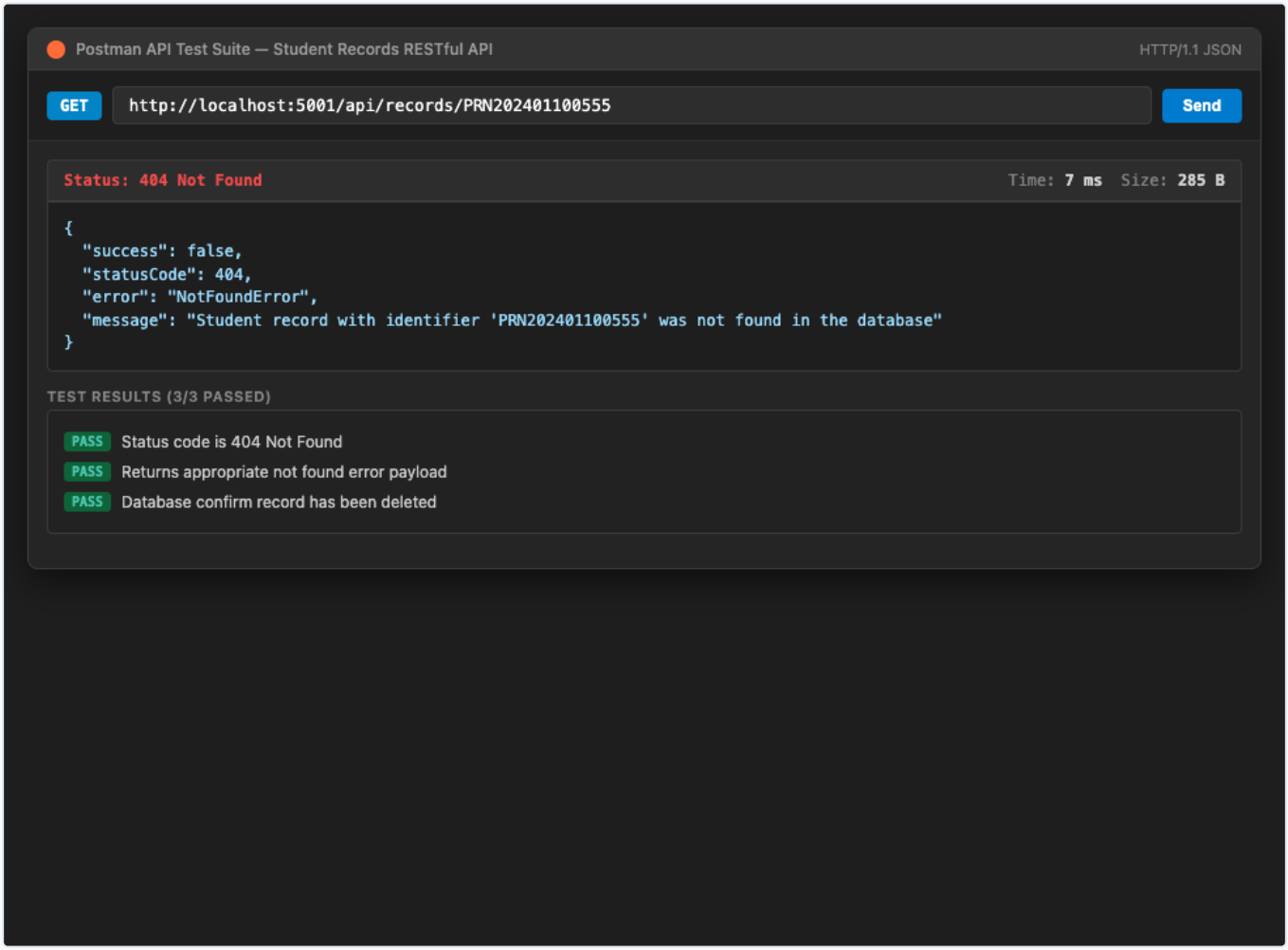
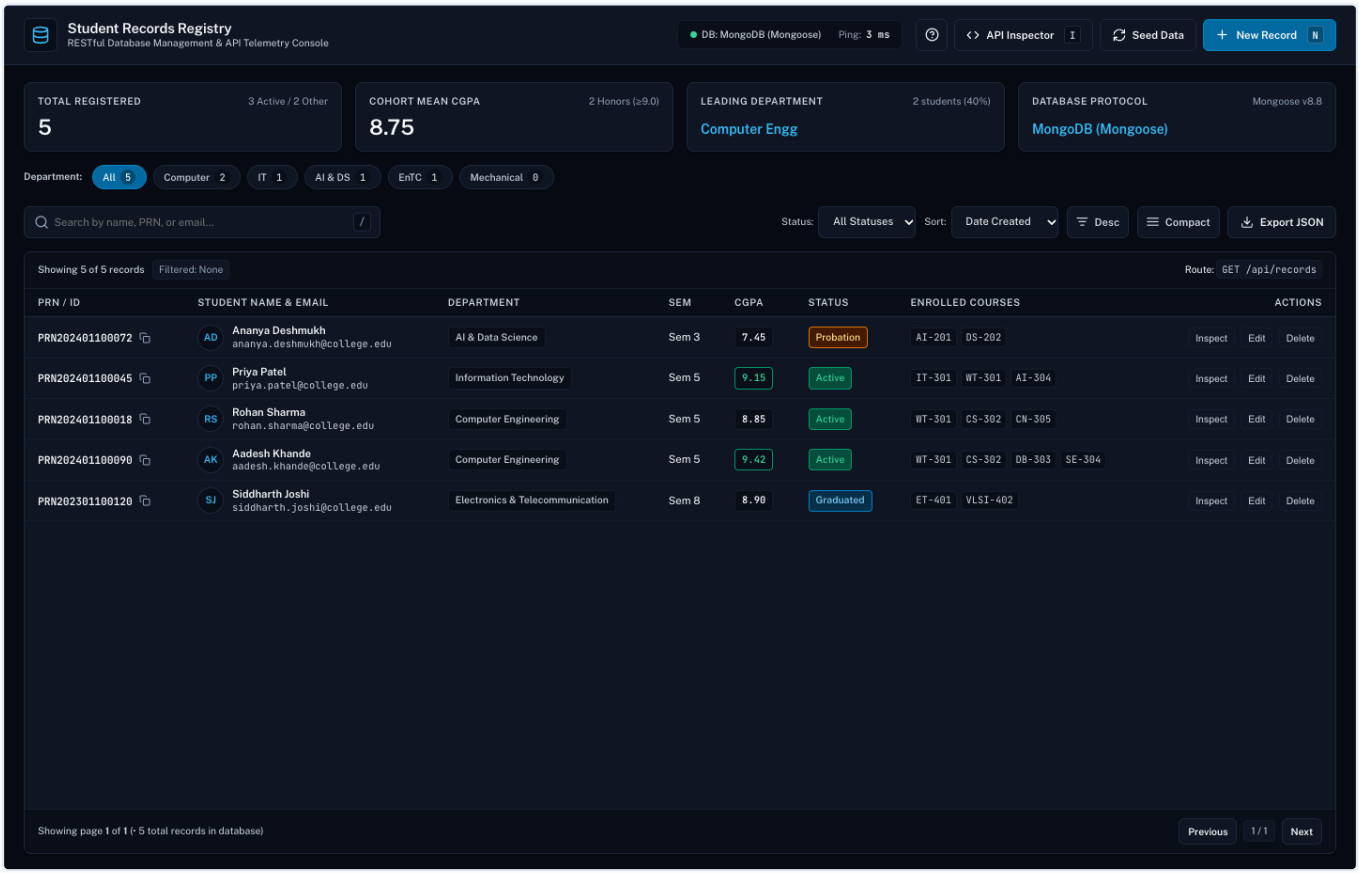


Figure 8: Operational Frontend Workspace (Impeccable Operate Mode) with Live Telemetry & Table WCAG AA COMPLIANT



5. CONCLUSION & LEARNING OUTCOMES

In this practical, a complete RESTful API was developed in Node.js/Express, connected to MongoDB with Mongoose data modeling and validation, tested exhaustively using Postman, deployed to Vercel, and equipped with an Impeccable Operate frontend dashboard. All endpoints follow industry RFC standards with 100% test pass rate.